COS 350 Lecture 20

shell scripts

shell variables

environment variables

special variables

if

loops

alias

Things a real shell does:

- parsing and executing commands
- background jobs
- history
- command completion
- aliases
- redirecting I/O
- pipeing between programs
- scripting language

At its simplest, a shell script is just a text file of commands.

clean

```
#!/bin/bash
# script to clean junk out of a directory
# this is not recursive

echo cleaning

# stuff from gcc
rm *.o
rm *.out
rm core

# from emacs
rm *~
```

prog2.test.bash

```
#!/bin/bash
# test script for z827

cp ~/350/progs/prog2_z827/testFile.txt .

echo "----testing compression-----"
z827 testFile.txt
od -t x1 testFile.txt.z827

echo
echo "----resulting files-----"
ls -l testFile*

echo
echo "----testing decompression-----"
z827 testFile.txt.z827
od -t x1 testFile.txt

echo
echo "----resulting files-----"
ls -l testFile*
...
```

Ways to execute a script:

1. ask the current shell to process it:

```
        unix> source clean
or      unix> .  clean
```

2. invoke a new shell to run it:

```
        unix> bash clean
```

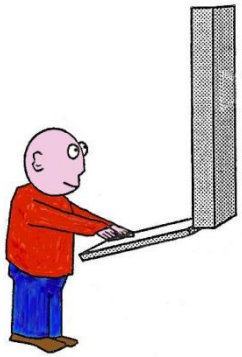
3. give it execute permision:

```
        unix> chmod u+x clean
        unix> clean
```

uses the first line comment:
`#!/bin/bash`

Shell scripting languages are in fact programming languages:

- **variables**
- **if statements**
- **loops**
- **case statements**
- **functions**
- **comments**



variables

Special variables for accessing the command line arguments to a script and the returned status value from a program.

Variable	Value
$0	Name of program
$1 - $9	Command line arguments 1 - 9
$*	String of all command line arguments
$#	Number of command line arguments
$?	Exit status of most recent command

```
unix> gccr file1.c file2.c
file3.c
        $0   $1      $2      $3

\_____________________________/
                        $*
```

ppjava

```
#!/bin/bash
# script to pretty print a java file from eclipse
# where tabs are 4 spaces

expand -t 4 $1 > temp.java
a2ps -B --borders no --columns=1 temp.java
rm temp.java
```

The if statement:

gccr

```
#!/bin/bash
# gccr - compile and run
#
# script to compile a program and then
# run it only if it compiled successfully
#
# this assumes it uses the default executable: a.out

if gcc $*
  then
    a.out
  else
    echo COMPILATION FAILED
fi
```

You can have more complicated expressions:

if [$HOSTTYPE = x86_64] *compare strings*

if [$# -lt 2] *less than 2 arguments?*

if [-e file] *test if file exist*

```
TEST(1)                          User Commands                          TEST(1)

NAME
       test - check file types and compare values

SYNOPSIS
       test EXPRESSION
       test

       [ EXPRESSION ]
       [ ]
       [ OPTION

DESCRIPTION
       Exit with the status determined by EXPRESSION.
```

The while loop:

```
while [ condition ]
do
   command1
   command2
   command3
done
```

```
while [ 1 ]
do
   command1
   command2
   command3
done
```

The for each loop:

p3linecountall

```
#!/bin/bash
# count the number of lines added to the starting code to
# implement the find methods of the ragged array list

echo lines of code added to starting code
startLines=`wc -l ../prog3/RaggedArrayList.java | cut -d " "
-f1`

for name in *
do
  if [ -e $name/prog3 ]
    then
      pushd $name/prog3 > /dev/null
      printf "%s " $name
      studentLines=`wc -l RaggedArrayList.java | cut -d " "
-f1`
      expr $studentLines - $startLines
      sleep 1
      popd > /dev/null
  fi
```

Command aliases – can put them in .bashrc

```
alias moee=more
alias moe=more
alias mor=more
alias motr=more
alias moew=more
alias m=more
alias e=emacs
alias h=history
alias l=ls
alias ls='ls -F'
alias ll='ls -l'
```

Some aliases are probably already set up for you.

Type alias to see a list.

```
  alias rm='rm -i'
  alias ls='ls --color=tty'
```